

Interview Preparation Guide

40 questions and answers across SQL, product analytics, pricing strategy, business rules, product metrics, LLM integration, and the trade-offs behind every design decision in this project.

SQL

Product Analytics

Product Management / Product Thinking

Pricing Strategy

Business Rules & Recommendation Engine

Product Metrics

Gemini Integration

Why SQL Instead of ML

Dataset Generation & Design Decisions

Trade-offs

Contents

40 questions across 10 categories

SQL	6 questions
Product Analytics	5 questions
Product Management / Product Thinking	5 questions
Pricing Strategy	4 questions
Business Rules & Recommendation Engine	4 questions
Product Metrics	4 questions
Gemini Integration	4 questions
Why SQL Instead of ML	3 questions
Dataset Generation & Design Decisions	3 questions
Trade-offs	2 questions

SQL

QUESTION 01

Walk me through how your recommendation engine works in SQL.

It's a single query against the Learners table with two layered CASE statements: one priority-ordered CASE picks the Recommended_Action (22 rules, evaluated most-important-first), and a second CASE picks the Recommended_Discount_Pct. Support-ticket and churn-risk rules are checked before any upsell rule, so a struggling learner is never pitched an upgrade before their issue is addressed.

QUESTION 02

Why did you use CASE statements instead of a stored procedure or multiple queries?

A single query keeps the logic auditable in one place - anyone can read top to bottom and see exactly why a learner got a given recommendation. Splitting it into multiple queries or a procedure would hide the priority ordering, which is the actual product decision here, not just the individual conditions.

QUESTION 03

How did you handle the fact that some learners could match multiple rules?

CASE statements in SQL only fire the first matching WHEN branch, so I used that directly: rules are ordered by business priority (satisfaction/churn risk first, then upsell, then loyalty, then default). That ordering is a deliberate design choice, not an accident of SQL syntax.

QUESTION 04

What's the difference between your Learners table and the two lookup tables?

Learners is the fact table - one row per learner with their behavior. Subscription_Plans and Learner_Segments are small dimension tables holding static attributes (price, target audience, typical renewal tendency) joined in via foreign keys. It's a light normalization that shows schema design awareness without overcomplicating a project this size.

QUESTION 05

How would you optimize these queries if the table had 10 million rows?

I added indexes on Learner_Segment, Subscription_Plan, and Renewed_Last_Cycle since those are the columns most queries filter or group on. At real scale I'd also consider partitioning by signup cohort or region, materializing the recommendation output nightly rather than computing it live, and profiling with EXPLAIN to catch any full table scans in the CASE logic.

QUESTION 06

Why use SUMIF/COUNTIFS in Excel instead of just pasting in values from SQL?

Formulas keep the workbook self-updating and auditable - if the underlying data changes, the summary and charts recalculate instead of going stale. It also let me catch a real bug: my first version stored numeric columns as text, which made SUMIF silently return 0 instead of erroring. AVERAGEIF did throw #DIV/0!, which is what caught it during recalculation testing.

Product Analytics

QUESTION 07

What's the single most interesting insight from your analysis?

Coupon users complete fewer courses (54% vs 66%) and renew less (58.8% vs 67.8%) than full-price learners. Discounting isn't purely upside - some of it is subsidizing learners who wouldn't have stayed at any price. That's a real trade-off a pricing team has to own, not just a vanity 'discounts drive growth' story.

QUESTION 08

How did you validate that your insights weren't just noise in a simulated dataset?

I designed the data generation with intentional segment-driven correlations - Power Learners engage more and renew more, Casual Learners engage less and churn more - and then re-ran every query against the actual generated CSV to confirm the numbers matched what I expected, rather than assuming. When I found a formula bug in the Excel workbook (SUMIF returning 0 silently), catching it early is exactly the kind of validation habit I'd apply to real data too.

QUESTION 09

Why does Pro generate more revenue than Standard despite having fewer subscribers?

Pro has 433 subscribers to Standard's 535, but at Rs 999 vs Rs 599 the price gap outweighs the volume gap - Rs 4,32,567 vs Rs 3,20,465. It's a reminder that subscriber count alone is a vanity metric; revenue-per-tier matters more for prioritizing which plan to optimize.

QUESTION 10

How would you present the churn-risk finding to a non-technical stakeholder?

I'd lead with the number, not the method: 'Casual Learners renew at 19.6% - roughly 1 in 5 - while Power Learners renew at 87%. That gap is bigger than anything driven by which plan someone's on.' Then I'd show the segment comparison chart, not the SQL behind it. Stakeholders want the implication, not the query.

QUESTION 11

What would you do differently if you had real user data instead of simulated data?

I'd validate the segment definitions themselves - in this project I assumed 6 segments exist and behave consistently, but real learners might not cluster that cleanly. I'd also want event-level data (timestamps of logins, purchases, plan changes) instead of a snapshot, since that's what's needed to measure true conversion and upgrade rates rather than the proxies I had to use here.

Product Management / Product Thinking

QUESTION 12

Why did you choose an EdTech subscription platform for this project?

It's a business model with clear, well-understood levers - plan tiers, renewals, discounts, churn - so I could focus the project on demonstrating pricing and retention thinking rather than explaining an unfamiliar industry. It's also directly relevant to Product Intern roles that touch subscription or monetization work.

QUESTION 13

How do you decide which learners to prioritize for outreach - upgrade candidates or churn risks?

Churn risk first. My recommendation engine deliberately checks support tickets and inactivity before any upsell rule. Losing an existing paying subscriber costs more than a missed upsell, and pitching an upgrade to someone with an unresolved support issue actively damages trust. That ordering was a product decision I made explicitly, not a default.

QUESTION 14

If you had to cut this project down to one slide for a recruiter, what would you show?

The recommendation-engine slide: 22 business rules, 14 pricing parameters, zero machine learning, with an explanation of why priority ordering matters. It's the piece that shows product judgment, not just SQL syntax - anyone can write a CASE statement, but deciding what order the rules fire in is the actual PM skill.

QUESTION 15

What's a limitation of this project you'd flag proactively in an interview?

The dataset is a snapshot, not a funnel log, so 'Conversion Rate' and 'Upgrade Rate' are proxy definitions I had to construct and label explicitly rather than measure directly. I'd rather say that upfront than have an interviewer catch it - it shows I understand the difference between what I built and what a production system would need.

QUESTION 16

How would you measure whether your recommendation engine actually works, if this were live?

I'd A/B test it - show the SQL-generated recommendation to a treatment group and nothing (or a generic message) to a control group, then compare renewal, upgrade, and discount-redemption rates between groups over one full billing cycle. Right now I can only say the rules are logically defensible, not that they causally improve outcomes - that's the honest gap between this project and a shipped feature.

Pricing Strategy

QUESTION 17

Why does the College Student segment get a 15% discount but Casual Learners don't?

College Students are price-sensitive but still engage well when they do convert (70%+ completion in the rule that triggers the discount), so a discount is likely to convert genuine interest. Casual Learners have low completion regardless of price, so a discount there would likely just subsidize disengagement rather than fix it - the rules engine treats 'price-sensitive' and 'low-intent' as different problems requiring different responses.

QUESTION 18

How did you decide on the four price points (Rs 299/599/999/1,499)?

Roughly geometric spacing so each tier feels like a meaningful step up, anchored to what each segment's stated budget sensitivity could support - Casual/College segments cluster on Basic and Standard, Working Professionals and Power Learners cluster on Pro and Premium. I'd want real willingness-to-pay research to validate these in production, but the spacing logic itself is defensible.

QUESTION 19

What would you tell a stakeholder who wants to just discount everyone to boost signups?

I'd point to the coupon-user data: discount users complete 12 percentage points less and renew 9 points less than full-price users. Broad discounting risks filling the funnel with lower-intent learners who churn anyway, which hurts the metrics that actually matter (renewal, LTV) even if signups look better short-term.

QUESTION 20

How does your engine avoid discounting learners who'd have paid full price anyway?

The discount rules are gated on behavior signals, not just segment membership - a College Student only gets the student discount if they're already showing 70%+ completion, and the loyalty discount only fires for learners who didn't renew but were clearly engaged (a likely price objection). Blanket segment-based discounting was a design choice I deliberately avoided.

Business Rules & Recommendation Engine

QUESTION 21

Why 22 rules instead of a simpler 5-10 rule system?

More rules let each one stay narrow and specific, which makes them individually easier to justify and test. A handful of broad rules would force compromises - e.g. one giant 'upgrade' rule covering every segment would blur why a Working Professional and a Career Switcher get flagged for different reasons. Narrow rules cost more to maintain but are more honest about the nuance in the data.

QUESTION 22

What happens when two rules could both apply to the same learner?

Only the first matching rule in priority order fires, by design. I ordered rules by business consequence: satisfaction and churn risk first, then power-user upgrade signals, then segment-specific offers, then loyalty rewards, then hygiene/cleanup rules, with a 'maintain/monitor' catch-all last. It's an explicit precedence list, not an accident.

QUESTION 23

How would you add a new rule to this system without breaking existing ones?

Insert the new WHEN clause at the correct priority position and re-run the engine against the full dataset to check the action-distribution table for anything unexpected - I did exactly this validation after building the engine, confirming every rule fires on real data with no dead branches. I'd also want a regression check comparing before/after recommendation counts per learner.

QUESTION 24

Isn't a rules engine like this eventually going to become unmaintainable as it grows?

Yes, and that's a real limitation I'd flag rather than pretend around. At 22 rules it's still fully human-auditable in one file. Past 50-100 rules, conflicts and overlaps get hard to reason about by eye, and I'd want a formal rule-conflict checker or a decision-table validation tool rather than just careful ordering.

Product Metrics

QUESTION 25

Walk me through how you calculated MRR and why it matters.

MRR is $\text{SUM}(\text{Monthly_Fee})$ across learners with at least one active month - it came out to Rs 11,78,400. It's the north-star subscription metric because it's the number that answers 'is the business predictable and growing,' and almost every other metric in this project (ARPU, renewal, upgrade rate) explains movement in MRR.

QUESTION 26

Your Conversion Rate and Upgrade Rate are marked as proxies - why, and what's the real definition?

Because this dataset is a single snapshot per learner, not an event log with a 'before recommendation' and 'after recommendation' state. I defined Conversion Rate as the renewal rate among coupon users (58.8%) and Upgrade Rate as the share of learners the SQL engine flags as upgrade candidates (29.9%) - both reasonable proxies, but I labeled them explicitly rather than presenting them as if measured the same way as MRR or Renewal Rate.

QUESTION 27

If you could add one more metric to this project, what would it be?

Customer Lifetime Value (LTV) segmented by learner type - it would directly connect the discount-sensitivity finding (coupon users renew less) to a dollar figure, which is what actually justifies a pricing policy change to leadership rather than just a percentage-point comparison.

QUESTION 28

Why track ARPU separately from MRR if they're derived from the same numbers?

MRR tells you the size of the business; ARPU tells you whether growth is coming from adding more users or extracting more value per user. At Rs 736.50 ARPU, knowing whether that's rising because of upgrades or falling because of new low-tier signups tells a very different growth story than MRR alone.

Gemini Integration

QUESTION 29

Why use an LLM at all if SQL already makes the decision?

Learners respond better to a natural, specific message than a table row. The LLM's job is narrow and low-risk: rephrase a decision that's already been made, in under 60 words, without changing it. That's a good LLM use case - low autonomy, high leverage on the output people actually see.

QUESTION 30

How do you keep Gemini from overriding or second-guessing the SQL recommendation?

The prompt explicitly instructs it not to change, second-guess, or add any pricing decision of its own - it's only allowed to explain the given action and reason. I also structured the Reason field itself with concrete numbers (e.g. '82% completion, 7.4 hrs/week') so there's nothing ambiguous left for the model to interpret or invent.

QUESTION 31

What happens if the Gemini API call fails for a learner?

The script catches the exception per-row and writes a placeholder message rather than crashing the whole batch - one failed call shouldn't block 1,599 successful ones. In production I'd also want a retry-with-backoff and a fallback to a templated (non-LLM) message so no learner ever sees a broken recommendation.

QUESTION 32

Could you have used a smaller/cheaper approach instead of an LLM API call per learner?

Yes - a template-based system (Mad-Libs style sentence with the numbers slotted in) would be nearly free and fully deterministic. I chose Gemini because the resume bullet specifically calls out LLM-assisted personalization, but I'd honestly tell an interviewer that for this specific task, a templating approach is a legitimate lower-cost alternative worth considering at scale.

Why SQL Instead of ML

QUESTION 33

Why not just train a classification model to predict churn instead of writing rules?

Two reasons: I don't have historical outcome data to train and validate a model against (this is a snapshot, not a labeled dataset with a true future outcome), and even if I did, a rules engine lets me change one specific decision in one specific rule without retraining or worrying about the model shifting behavior elsewhere. For an early-stage product, that transparency and quick-iteration property matters more than the marginal accuracy a model might add.

QUESTION 34

Isn't SQL just a worse, manual version of what ML would do automatically?

For a mature product with lots of labeled outcome data, yes - ML would likely outperform hand-written rules eventually. But 'automatically' is doing a lot of work in that question: ML needs feature engineering, retraining, monitoring for drift, and a way to explain individual decisions to stakeholders and possibly regulators. SQL rules need none of that. The right tool depends on what data and organizational maturity actually exist, not which one sounds more impressive.

QUESTION 35

When would you actually recommend switching this project's SQL rules to an ML model?

Once there's enough real outcome data (a few thousand labeled renewal/churn/upgrade events with a clean pre/post-recommendation signal) and once the SQL rules' accuracy has plateaued despite iteration. At that point I'd frame it as an upgrade, not a rewrite - keep the rules as a transparent baseline/fallback and use the model to flag the harder edge cases the rules miss.

Dataset Generation & Design Decisions

QUESTION 36

How did you make sure your simulated dataset wasn't just random noise?

I built segment-specific behavior ranges directly into the generator - e.g. Power Learners get sampled from a higher weekly-hours and completion-rate range than Casual Learners - so correlations exist by construction. I then validated this by running the actual SQL analysis and confirming the renewal-rate gradient across segments matched what I'd designed (19.6% up to 86.7%, in the expected segment order).

QUESTION 37

Why weight the segments unevenly (28% College Students down to 6.6% Power Learners) instead of an even split?

It mirrors how EdTech platforms actually acquire users - broad, price-sensitive segments dominate signups while the highest-value, most-engaged segment is naturally the smallest. An even split would have been easier to generate but would have misrepresented the acquisition funnel a real platform faces.

QUESTION 38

What's a limitation of your data generation approach you'd want to fix?

Payment_Mode was assigned fully at random with no segment logic, so any 'insight' drawn from it would be noise, not signal - I flagged this explicitly rather than let it slip into the analysis. It's a reminder that every column in a simulated dataset needs an intentional generation rule, or it shouldn't be used to support a claim.

Trade-offs

QUESTION 39

What's the biggest trade-off you made in this project, and why?

Interpretability over sophistication - choosing a fully SQL rules engine over any ML component, even though ML might eventually perform better with enough data. I made that trade because the project's whole value proposition (defensible, auditable pricing decisions for a new product with no historical data) depends on every decision being traceable to a named, explainable rule.

QUESTION 40

If you had two more weeks on this project, what would you prioritize?

Turning the recommendation engine into something testable end-to-end: simulate a 'before' and 'after' state for a subset of learners, run a mock A/B test comparing rule-driven recommendations against a no-recommendation control, and report real (if simulated) lift numbers instead of just the current-state metrics I have now.